

N8N Automations

A Deep Dive

Portfolio explainer document
Prepared for Salman Adnan

Generated 2026-07-07

What this document is. A beginner-safe, exhaustive walkthrough of the `n8n-automations/` folder in this Portfolio: 59 workflow-automation builds made in n8n for Digitalise Agency. It explains what n8n is, walks through several real automations in detail with diagrams of their actual node graphs, and covers the curation process and the honest defects that process turned up. Every fact below comes from the top-level `README.md`, the curation record `N8N_WORKFLOW_PROPOSALS.md`, and a close reading of 14 individual automation `READMEs` spanning every category in the collection (lead-gen/scraping, AI content pipelines, agent/orchestrator systems, business ops/CRM, RAG/vector-store ingestion, and small utility bots). Anything not directly stated in those files is marked explicitly as inferred. This is a working document for the owner, not website content.

Contents

1	Before anything else: what is n8n?	3
1.1	Core vocabulary	3
1.2	How to read a diagram in this document	4
2	The collection at a glance	4
3	Worked examples	5
3.1	Google Maps to Email Scraper — lead generation	5
3.2	Ultimate Personal Assistant System — orchestrator plus four sub-agents	6
3.3	WhatsApp AI Sales Lead System — three subsystems, and where the wiring breaks	7
3.4	RAG Ingestion Pipeline — ingest documents, then query them	8
3.5	Social Media Content Generator — the collection’s largest workflow	9
3.6	CRM Lead to Stripe Invoice — the simplest worked example	10
3.7	Cold Email Outreach Sequence System — nine workflows, one orchestrator	11
3.8	Deep Research Report Generator — fan-out, then a spreadsheet as the meeting point	11
4	Curation, scrubbing, and verification methodology	12
4.1	Naming and anonymization	13
5	Honestly-documented real defects, seen across the collection	13

6	What remains blocked, and why	14
7	Closing notes and honest limits of this document	15

1 Before anything else: what is n8n?

n8n (pronounced “n-eight-n”, short for “nodemation”) is a **visual workflow automation tool**. Instead of writing a script line by line in a text editor, you build an automation as a **node graph**: a canvas of boxes (**nodes**), each doing one job, connected by lines (**connections**) that show which node’s output feeds which node’s input. Running the workflow means data flows through that graph, node by node, left to right (by convention, not by rule).

This is still engineering work. Every workflow in this collection chains together triggers, API calls, LLM steps, and data transforms into a working system — the same design thinking as writing code, just expressed as a diagram instead of text. A sloppy node graph can have the exact same bugs as sloppy code: dead branches, race conditions, hardcoded values, missing error handling. Several of those turn up later in this document, honestly, because they were found in the real exported files.

1.1 Core vocabulary

Trigger node — the node that starts a workflow running. Every workflow needs at least one. Common triggers seen in this collection: a **manual trigger** (a person clicks “Execute Workflow” in the n8n editor), a **schedule trigger** (runs on a cron-style timer, e.g. every Tuesday at 1:02am), a **webhook** (see below), a **form trigger** (an n8n-hosted web form that starts the workflow on submission), and app-specific triggers like a **Telegram trigger** (fires when someone messages a bot) or a **Google Drive trigger** (fires when a file changes in a watched folder).

Webhook — a URL that some other system can send an HTTP request to. When that request arrives, it starts the workflow. This is how, for example, a ClickUp CRM automation can tell an n8n workflow “a deal was just marked won” the instant it happens, instead of n8n having to poll and ask.

Connection — the line drawn between two nodes on the canvas. It means “when this node finishes, hand its output to that node.” n8n has more than one *kind* of connection: a plain **main** connection carries ordinary data from one node to the next; special connection kinds (`ai_tool`, `ai_memory`, `ai_languageModel`, `ai_embedding`, `ai_textSplitter`, `ai_document`) are used only by AI-related nodes to attach a model, a memory store, a tool, or a document-processing helper to an agent or a vector-store node. A node can be fully configured and visually sit right next to another node on the canvas and still have *zero* connections to it — several real examples of exactly this mistake (a fully built node nobody ever calls) turn up in Section 5.

Sub-workflow / orchestrator pattern — instead of building one giant workflow that does everything, a workflow can call *another whole workflow* as if it were a single step, the same way a program calls a function. The calling workflow is often called an **orchestrator**: it does little work itself, mostly deciding which sub-workflow to call and in what order. The called workflow is entered through a special trigger, `executeWorkflowTrigger` (“When Executed by Another Workflow”), meaning it only runs when something else calls it, never on its own. This collection has several orchestrator systems; Section 3.2 and Section 3.7 are full worked examples.

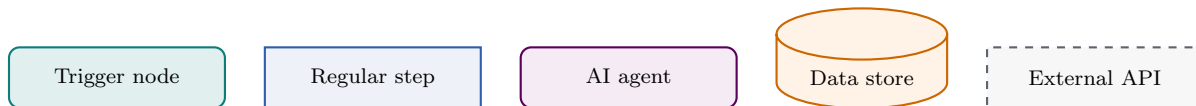
AI agent node — a node type (in this collection, always from the `@n8n/n8n-nodes-langchain` package) that wraps a large language model (LLM) in a loop: give it a goal and a set of **tools**

(other nodes it's allowed to call), and the agent node decides, turn by turn, which tool to call and with what input, until it has an answer. This is different from just calling an LLM once for a single text output — an agent can call a calendar tool, then an email tool, then reply, all in one execution, driven by the model's own reasoning rather than a fixed sequence hardcoded by the workflow author.

Vector store / RAG pipeline — **RAG** stands for Retrieval-Augmented Generation. Instead of an LLM answering purely from what it was trained on, a RAG pipeline first breaks real documents (PDFs, spreadsheets, web pages) into small **chunks**, converts each chunk into a list of numbers (an **embedding**) that captures its meaning, and stores those embeddings in a **vector store** (a database built for fast “find the most similar chunks to this query” lookups — this collection uses **Supabase** with the **pgvector** extension). When a user later asks a question, the pipeline embeds the question the same way, finds the closest-matching chunks in the vector store, and hands those chunks to the LLM as extra context before it answers. This collection has both a standalone RAG ingestion pipeline (Section 3.4) and RAG built into a larger agent system (Section 3.3).

1.2 How to read a diagram in this document

Every diagram below is a simplified, boxes-and-arrows redraw of the *real* node graph, using the *real* node names and order found in the workflow's JSON export and its README, not an invented or idealized version. A large workflow (the biggest in this collection has 190 nodes) is summarized to its structurally important nodes; the prose around each diagram says plainly what was collapsed. The shape legend used throughout:



2 The collection at a glance

`n8n-automations/` holds 59 individually documented automation folders, each with an exported workflow JSON (importable directly into n8n), a README explaining what it does and its honest limits, a `CREDENTIALS.md` listing exactly which external accounts a re-user would need, and a `LICENSE` file. This mirrors how the rest of this Portfolio is organized, applied to n8n's visual format instead of a code repository.

The automations span several recognizable categories. This is not an official taxonomy from the source material, it is a reading of the 59 folder names and the 14 READMEs read closely for this document, grouped for orientation:

- **Lead generation and scraping** — e.g. `google-maps-to-email-scraper`, `linkedin-extractor-from-google`, `instagram-link-finder-outreach`, `deep-multiline-icebreaker-system`.
- **AI content generation pipelines** — e.g. `social-media-content-generator`, `cluster-seo-post-generator`, `trend-seo-post-generator`, `auto-video-generator`, `faceless-shorts-system`, `fb-ad-creative-generator`, `text-to-image-generator`.
- **Agent and orchestrator systems** — e.g. `ultimate-personal-assistant-system` (an orchestrator plus 4 callable sub-agents), `upwork-ai-agent-pipeline` (an orchestrator plus 3 sub-

workflow tools), `whatsapp-ai-sales-lead-system`, `content-business-chat-agent`.

- **RAG / vector-store ingestion** — e.g. `rag-ingestion-pipeline`, `tutorial-rag-chat-agent`.
- **Business operations and CRM** — e.g. `crm-lead-to-stripe-invoice`, `expense-tracker-to-sheets`, `calcom-trigger-workflow`, `booking-time-utility-set`.
- **Outreach and email sequencing** — e.g. `cold-email-outreach-sequence-system`, `gmail-outreach-follow-up`, `subscriber-email-sequence`, `newsletter-automation`.
- **Small utility and messaging bots** — e.g. `telegram-checklist-generator`, `telegram-joke-agent`, `telegram-translator-bot`, `uuid-generator-utility`, `website-screenshot-utility`, `daily-health-tip-bot`, `birthday-greetings-automation`.

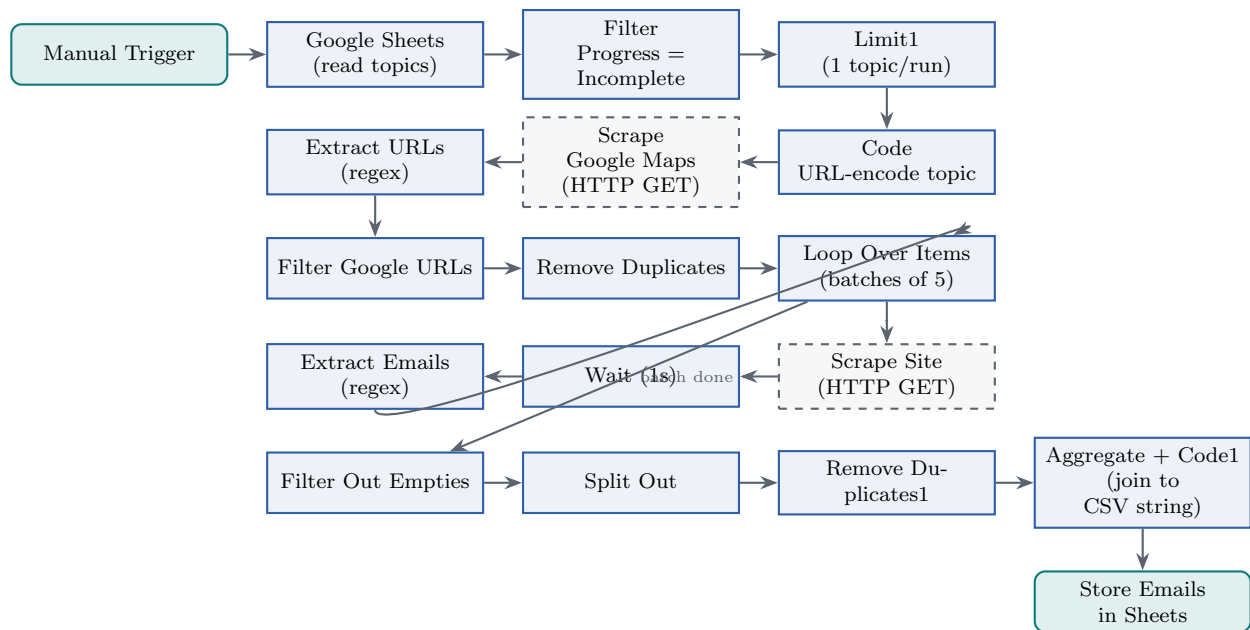
This document does not attempt node-by-node coverage of all 59 automations; that would bury the reader in repetition rather than build understanding. Section 3 covers eight automations in real depth, chosen to represent every category above and to include the collection’s largest workflow (190 nodes), its most elaborate multi-file orchestrator system, and its most honestly self-critical set of disclosed defects. Sections 4 and 5 then step back and describe the collection as a whole: how it was curated, what was found broken, and what remains unbuilt.

3 Worked examples

Each example below is read from the automation’s own README and, for architecture detail, its stated node graph. All node names quoted are the workflow’s real internal names as exported from n8n.

3.1 Google Maps to Email Scraper — lead generation

What it does. Reads pending search topics (e.g. “Calgary dentists”) from a Google Sheet, scrapes Google Maps search results for matching businesses with no Maps API key at all (a plain HTTP GET against `google.com/maps/search`), visits each business website it finds, extracts email addresses from the raw HTML with a regex, and writes the deduplicated results back to the same sheet.

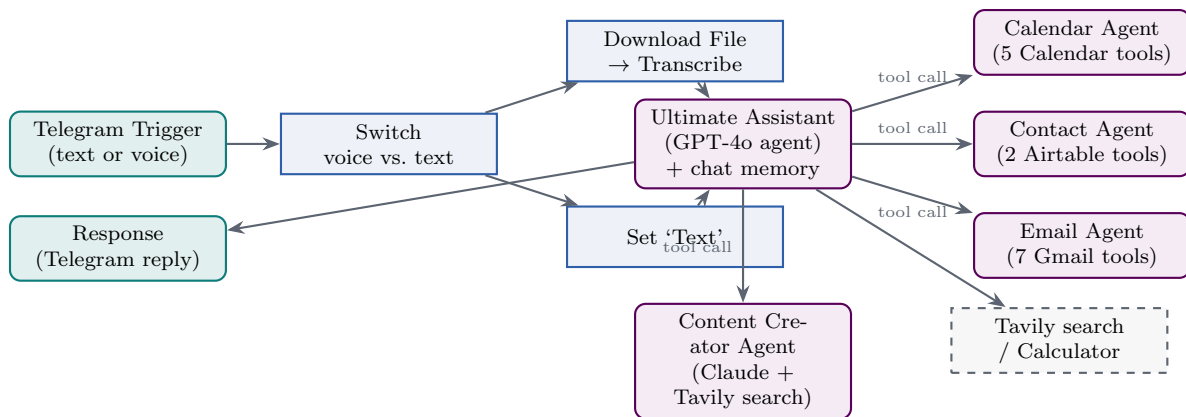


Reading the loop. `Loop Over Items` is n8n’s `splitInBatches` node used as an iterate-and-fetch loop: its “still looping” output feeds the per-item scrape-wait-extract chain and then feeds *back into itself*, while its “batch finished” output only fires once every website in the batch of 5 has been visited, carrying the accumulated results forward to the email-deduplication steps. This two-output loop pattern (loop back vs. done) recurs across many automations in this collection wherever a workflow needs to process a list of items one at a time.

Real, disclosed defects. The README documents these plainly: Google Maps has no stable HTML structure, so the regex-based URL extraction has no fallback if Google’s markup changes; `Limit1` caps every manual run to exactly *one* topic (n8n’s default when `maxItems` is left unset), so working through a backlog means running the workflow once per topic; deduplication only happens within a single run, so re-running an already-completed topic re-scrapes from scratch; and scraping `google.com/maps/search` directly by HTTP request is not something Google’s terms of service condone — the README states this outright rather than glossing over it.

3.2 Ultimate Personal Assistant System — orchestrator plus four sub-agents

What it does. A five-workflow system built around one Telegram bot. A user sends text or a voice note; a top-level orchestrator agent decides what kind of task it is and calls exactly one of four specialist sub-agents to actually do the work: calendar, contacts, email, or blog-content creation. This is the clearest orchestrator pattern in the whole collection, and it is a deliberate design choice stated in the README: one giant agent with every tool bolted on tends to confuse an LLM about which tool to pick, so the work is split into a router that only routes, and specialists that only know their own domain.

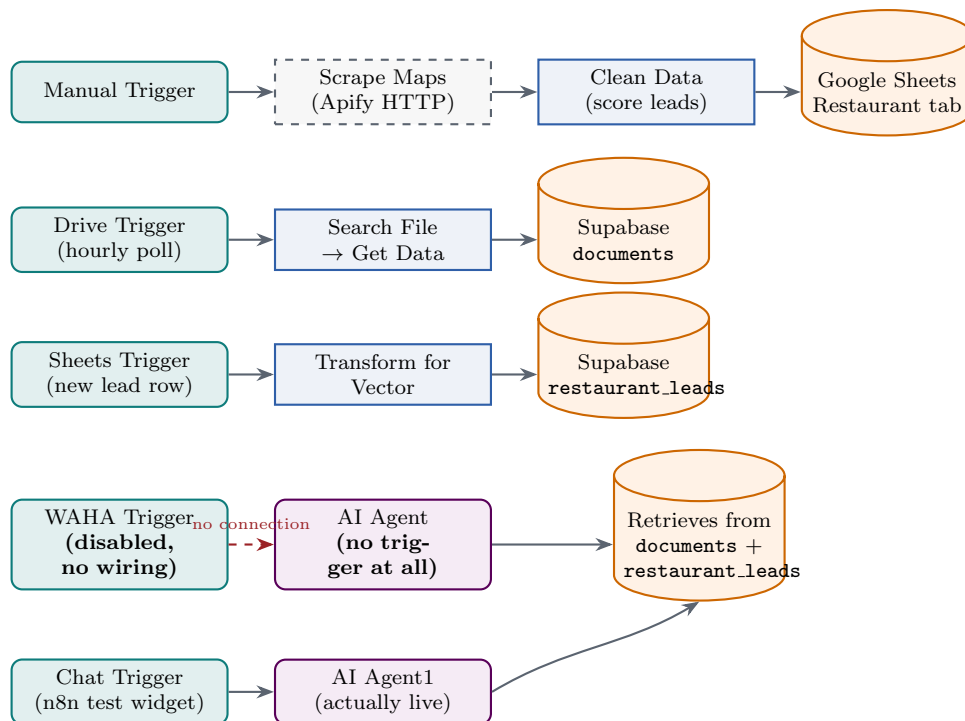


The “Call n8n Workflow Tool” pattern. Each of the four boxes on the right is a separate workflow file, entered only through an `executeWorkflowTrigger` (“When Executed by Another Workflow”), meaning it never runs on its own; it is attached to the orchestrator as a `toolWorkflow` node, which is n8n’s way of letting an agent call a whole other workflow the way it would call any other tool. The orchestrator passes a single free-text `query` string and gets back the sub-agent’s final answer. A built-in cross-agent rule lives in the orchestrator’s system prompt: before sending an email or creating a calendar event with an attendee, it must call the Contact Agent first to resolve a name into an email address.

Real, disclosed defects. This system’s README is unusually candid about where prompt-only guardrails are not the same as real ones: the “get the ID before you delete/reply” rule for the Calendar and Email agents lives entirely in prompt text, not in any IF node checking that a real ID was found, so nothing in the graph stops the model from guessing one; contact resolution before emailing is advisory, not enforced; every sub-agent’s failure collapses to one identical apology string regardless of whether the real cause was a bad OAuth token or a rate limit; the voice transcription path has no error branch for a failed download or empty transcription; and the Content Creator agent runs on Anthropic Claude while every other agent in the same system runs on GPT-4o, with no explanation given anywhere in the workflow for the inconsistency. The author states plainly that none of this was exercised against real credentials — the write-up reflects a reading of the exported graphs, not an observed live run.

3.3 WhatsApp AI Sales Lead System — three subsystems, and where the wiring breaks

Naming note, stated first because it matters. The live n8n workflow this folder is built from is literally named “*Cold Calling Automation*” in its own exported JSON. There is no phone dialer or telephony node anywhere in its 48 nodes. The folder name and README describe what the workflow actually *is*: a Google Maps lead scraper, a RAG ingestion pipeline into Supabase, and a WhatsApp-facing AI agent, wired onto one canvas.

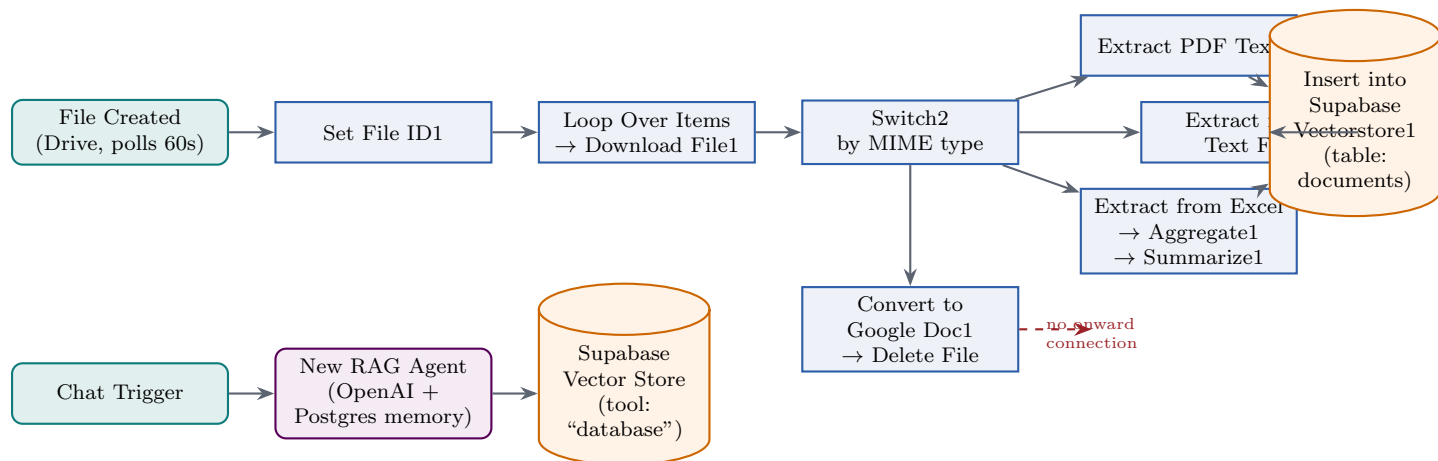


What actually runs. The scraping and ingestion halves (top two rows) are fully wired and would run as exported. The WhatsApp-facing half is designed but broken: **WAHA Trigger** is disabled *and* has no outgoing connection at all; **AI Agent** (the instance whose prompt and memory key are literally written against WAHA’s WhatsApp payload shape) has *no incoming trigger of any kind*, so as exported it cannot run, full stop. The node that is actually connected to a live trigger is a second, parallel agent, **AI Agent1**, fed by n8n’s own built-in test-chat widget, not WhatsApp. Two Cohere reranker nodes also sit on the canvas near the retrieval tools with zero connections anywhere in the graph — present, fully configured, and doing nothing.

This is a genuinely useful lesson in how to read an n8n export: a node being present on the canvas, fully configured, with a model, memory, and tools all wired in through the AI-specific connection types, still tells you nothing about whether it ever runs. That is a separate question, answered only by tracing the plain **main** connection back to a trigger. This system has one agent that looks completely finished and cannot run, and one that is a test stand-in but is the one actually live.

3.4 RAG Ingestion Pipeline — ingest documents, then query them

What it does. Watches a Google Drive folder, extracts text from whatever lands in it (PDF, plain text, Google Docs, Excel, Word), chunks it, embeds it with OpenAI, and writes the vectors into a Supabase **documents** table via a **match_documents** Postgres function. A second half of the same workflow exposes a chat-triggered agent that queries that same table to answer questions.

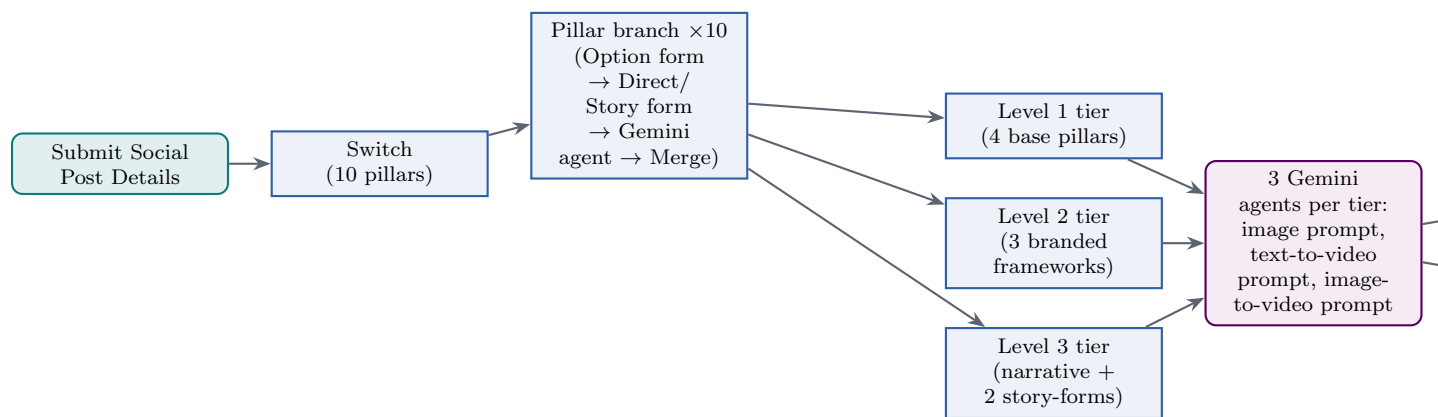


The unresolved Word-document branch. This is one of the clearest examples in the whole collection of a real, verifiable gap: the Word-document branch converts a `.doc/.docx` file to Google Docs format and deletes the original, but that branch has *no connection forward* to extraction or insertion anywhere in the exported graph. The likely intent, stated in the README as inference and not fact, is that creating the Google Doc copy fires a new Drive “file created” event, which re-enters the same trigger and gets extracted on a second pass. That dependency on the trigger re-firing is invisible in the graph itself; nothing enforces or documents it there.

A second honest disclosure: the entire “File Updated” half of the pipeline (which would delete stale vector rows and re-embed a changed file with a bumped version number) is present, fully built, and simply **disabled** — both the trigger node and the workflow’s own **active** flag are off. A workflow that looks feature-complete on paper (create and update handling, versioning, stale-row deletion) is actually only running the create path unless someone deliberately flips both switches on.

3.5 Social Media Content Generator — the collection’s largest workflow

Scale. 190 nodes in a single workflow file, the largest in the collection. A form submitter picks one of ten copywriting “pillars” (e.g. “Pain Point & Solution”, or a branded framework called “The Art of War”), then a tone (“Direct Based” or “Story Line Based”), fills in pillar-specific facts, and the workflow writes the post copy, an image prompt, a video prompt, and renders an actual image and video clip, all returned as the form’s own response.

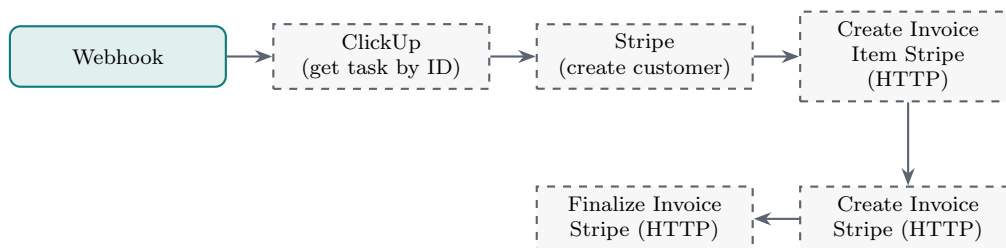


Why the diagram is a tier summary, not a node count. Tracing all 190 nodes on one page would be unreadable and would not teach anything a summary can't. The real structure that matters: ten fully copy-pasted per-pillar branches (there is no native way in n8n to loop a form wizard over a list of content types) collapse into three shared image/video pipelines by tier, so the expensive part (image and video generation) is not duplicated ten times over. That fan-in is visible directly in the workflow's `Merge12` / `Merge1` / `Merge13` nodes.

Real, disclosed defects. The node names show real seams: several shared downstream nodes still carry the name of whichever pillar was built first (the shared Level 1 image-prompt agent is still named "Pain Point - Text-To-Image" even though it also processes three other pillars' output through the same merge). One form option ("Narratives & Mythological Frameworks") is grouped visually with the branded frameworks in the dropdown but its output actually flows into the Level 3 (narrative) tier in the connections graph — a real, visible mismatch between label and wiring, not confirmed as intentional or accidental. The Novita.ai video-render poll loop (`Wait` → `httpRequest` → `If`) has no visible cap on retry count, so a render that never reports success appears to wait indefinitely. There is not one single error-handling branch anywhere across all 190 nodes.

3.6 CRM Lead to Stripe Invoice — the simplest worked example

What it does. A webhook fires when a ClickUp CRM deal is marked "Lead Won." Six nodes, one straight chain, no branching, turn that into a finalized, emailed Stripe invoice with no human touching Stripe.

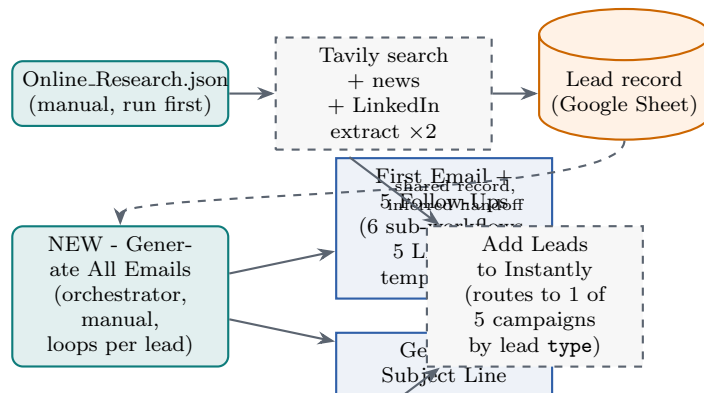


Worth noting as a teaching example. Customer creation goes through n8n's native Stripe node, but the invoice item, the invoice itself, and finalizing it all go through raw `httpRequest` nodes calling Stripe's REST API directly, reusing the same Stripe credential via `predefinedCredentialType`. This is a real, practical n8n pattern: a native node can be paired with hand-rolled HTTP calls against the same API when the native node doesn't cover an operation (here, invoicing), without configuring a second credential.

Real, disclosed defects, all six numbered plainly in the README. Custom fields are read off the ClickUp task by raw array index (`custom_fields[19]` for email, `custom_fields[5]` for the deal amount), so reordering fields on the ClickUp list would silently shift which value lands in which field, with no error thrown; the dollars-to-cents conversion (`value * 100`) assumes a clean two-decimal USD number with no currency or rounding check; there is no status check on the webhook itself confirming the payload really represents a won deal; there is no duplicate protection, so a retried webhook creates a second customer and a second invoice; the invoice line description is the hardcoded literal string "1st Deposit" for every invoice regardless of deal type.

3.7 Cold Email Outreach Sequence System — nine workflows, one orchestrator

What it does. Turns a raw lead (name, company, LinkedIn URL) into a full six-touch cold email sequence pushed into an Instantly.ai sending campaign: one research workflow, five copy-generating workflows (first email plus four/five follow-ups), a subject-line workflow, a delivery workflow, and one orchestrator that loops over a batch of leads and drives the rest.

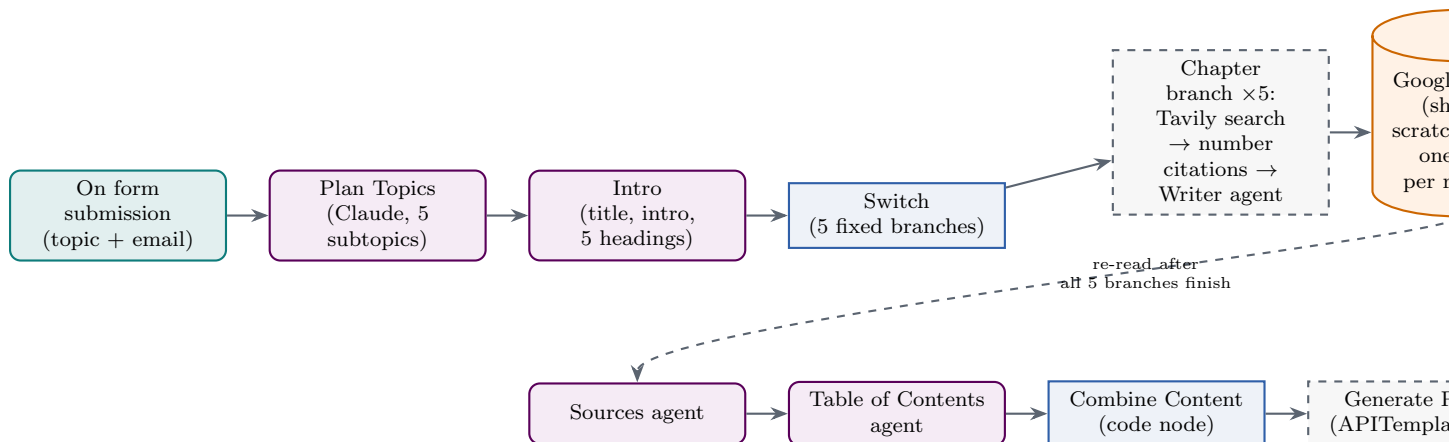


The inferred handoff. The research workflow and the orchestrator are never directly connected in the JSON — there is no node link between them. The README is explicit that the dependency is inferred, not observed: every copywriting workflow’s LLM prompt reads field names (`general_search`, `news_search`, `person_linkedin_scrape`, `company_linkedin_scrape`) that only the research workflow produces, which means the research pass is expected to run first and write its output back into the same lead record before the orchestrator runs, even though nothing in the graph enforces that ordering.

Real, disclosed defects. Each of the six copywriting workflows independently duplicates the same “run 5 parallel LLM attempts and pick a usable one” pattern and its own output-cleaning code, with no shared sub-workflow, so a prompt fix has to be copied by hand into six files; the orchestrator file still contains nine *disconnected*, orphaned `executeWorkflow` nodes left over from an earlier version, with no incoming or outgoing connections, dead weight that was never cleaned up; nothing anywhere in the ten files checks whether a lead has already been researched or emailed before running again, so re-running a batch would duplicate Instantly.ai leads.

3.8 Deep Research Report Generator — fan-out, then a spreadsheet as the meeting point

What it does. One form submission (a topic and an email address) is broken into five subtopics by an LLM, researched in five parallel branches via Tavily search, written up as five HTML chapters with numbered inline citations, stitched into one report with a title page, table of contents, and sources list, rendered to a PDF, and emailed to the submitter.



The spreadsheet as a synchronization mechanism. Five research-and-write branches finish at different times because each one calls an LLM. Rather than trying to hold five in-flight branches in memory and merge them directly, the workflow writes each branch’s content to its own columns in one shared Google Sheet row, and a `Merge2` node configured with `numberInputs: 5` waits for all five branch writes before the final-assembly steps re-read the completed row. This is a practical, real workaround for coordinating asynchronous branches in a tool that has no first-class “wait for these five parallel things” primitive beyond a merge node counting inputs.

Real, disclosed defects. None of the five Tavily search nodes have retry-on-fail configured; if any one of the five parallel branches’ search call fails, that branch’s sheet write never happens, and the `numberInputs: 5` merge stalls forever waiting for a fifth input that will never arrive, with no visible failure surfaced anywhere. Rows are matched by the literal search-topic text, not a run ID, so two submissions of the same topic would collide and could corrupt a still-in-flight report. Citation numbering offsets across the five chapters are hardcoded literals (+1, +6, +11, +16, +21), which only line up correctly if Tavily returns exactly five results per subtopic every time.

4 Curation, scrubbing, and verification methodology

This is not a from-scratch build log; it is a curation and documentation pass over work that already existed on Digitalise Agency’s live n8n instance and in on-disk backup exports. The process, stated in the top-level README and the working record `N8N_WORKFLOW_PROPOSALS.md`:

1. **Survey.** Every workflow on the live n8n instance and every workflow file on disk was inventoried in `N8N_WORKFLOW_PROPOSALS.md`, organized into three tiers: **Tier 1 – Live agency workflows** (the primary list, largest tier), **Tier 2 – On-disk workflows not found live** (two entries: an Instagram scraper variant and a LinkedIn publish workflow that exist as files but have no matching live n8n workflow), and **Tier 3 – Downloaded reference material** (course kits, community-template repositories, and link lists that are explicitly *not* the agency’s own work, listed for disclosure, not for inclusion).
2. **Approve or reject, line by line.** Each Tier 1/2 entry in the proposals document carries an explicit approve/reject mark. Near-identical test, demo, or duplicate copies of the same underlying workflow (for example, the Cold Email system’s original build plus its rebuilt “NEW -” prefixed version) are grouped into one line with one decision rather than one line each.
3. **Scrub.** Approved workflows had personal information and live credentials removed from the

exported JSON before being placed in this folder — for example, the Google Maps scraper’s README notes directly that both copies of its source file had a Google Sheet document ID and a personal email address, embedded in the file’s project metadata, replaced with placeholders.

4. **Document.** Each approved automation got its own folder with a workflow JSON, a README (What it is, Why it exists, Features, Architecture, Setup, Usage, Challenges, What I learned, What I’d do differently), a `CREDENTIALS.md` listing every external account needed and which node needs it, and a `LICENSE`.
5. **Verify, honestly, at the level actually possible.** Every README in this collection states plainly, usually in its own closing note, that it was written from reading the exported node graph and its parameters, *not* from a live execution against real credentials. Several READMEs go further and flag specific behavior that “was not verified here” or “was not run and observed” — for example, whether Apify’s dataset response really does flow one reel at a time through the rest of the Instagram Reel Analyzer’s chain is stated as standard n8n behavior but explicitly not confirmed by an actual run.

4.1 Naming and anonymization

A few workflows in the live instance are named after real clients, or named after their live n8n internal id rather than what they actually do (the WhatsApp Sales Lead System’s “Cold Calling Automation” name in Section 3.3 is the clearest example). Where that was the case, the folder name and README describe the automation by its function, not by a client’s name or an unrelated tutorial creator’s handle, and any naming mismatch or third-party lineage (course-derived, community-template-derived) is disclosed honestly in the individual README. Several automations are explicitly flagged in `N8N_WORKFLOW_PROPOSALS.md` as likely derived from a paid course or a public community template rather than an original design — for example, the Cluster SEO Post Generator’s own on-disk filename carries a “_SKOOL” suffix, naming the community platform its template came from, and its README states this directly rather than presenting the workflow as an original build.

5 Honestly-documented real defects, seen across the collection

The individual worked examples above already surfaced many specific defects. This section groups the *recurring* categories of real, disclosed problems that show up across multiple automations, because the pattern matters more than any single instance of it.

Fully-wired nodes with no trigger. A node can have a model, a memory, and a full set of tools attached through every AI-specific connection type and still never run, because the plain `main` connection back to a trigger is a separate, easy-to-miss layer. The WhatsApp Sales Lead System’s `AI Agent` (Section 3.3) is the clearest case: fully dressed, zero incoming connections, cannot execute as exported.

Disabled triggers and inactive workflows shipped as if complete. The RAG Ingestion Pipeline’s update-handling half (Section 3.4) is fully built, including versioning and stale-row deletion, but both its trigger node and the workflow’s own `active` flag are off. The Telegram Checklist Generator’s only trigger is saved disabled in its export, so a straight import-and-

activate silently does nothing until someone notices.

Dead-end branches with no forward connection. The RAG pipeline’s Word-document conversion branch (Section 3.4) converts a file and deletes the original but never connects onward to extraction; the WhatsApp system’s WAHA trigger is disabled *and* unconnected at the same time. Both are visible directly in the `connections` object of the exported JSON, not inferred from behavior.

Duplicated logic that can silently drift. The WhatsApp system’s two ingestion paths each rebuild their own version of the same business-summary string in separately written code nodes; the Cold Email system’s six copywriting workflows each duplicate the same five-attempt generation pattern and its own cleaning regex with no shared sub-workflow; the Ultimate Personal Assistant’s four sub-agents are separate, independently maintained prompt strings. Every one of these is a place where a fix made in one copy and forgotten in another produces quietly inconsistent behavior.

Hardcoded values standing in for configuration. The Google Maps scraper and the Instagram Reel Analyzer both hardcode a spreadsheet ID into more than one node instead of centralizing it; the CRM-to-Stripe workflow reads ClickUp custom fields by raw array index rather than field ID; the Telegram Checklist Generator and the Expense Tracker both reply to one fixed, hardcoded Telegram chat ID rather than the sender’s actual chat, which limits either bot to a single user as built.

Missing error handling around paid external APIs. None of the 190 nodes in the Social Media Content Generator include an error-trigger, retry, or fallback branch. The Faceless Shorts System chains six or more paid generation APIs (image, video, audio, render) end to end with fixed-duration `Wait` nodes standing in for real polling, and several of its HTTP calls to PiAPI, Runway, and ElevenLabs ship with no authentication configured at all in the exported file — a genuine gap, not a design choice, that would need fixing before those calls could succeed.

LLM output trusted without a validation layer. The Expense Tracker’s running total is computed by the LLM re-reading and summing spreadsheet rows itself, with no deterministic Aggregate node checking its arithmetic. The Deep Research Report Generator’s structured planning calls rely on the model returning an exact JSON key schema with no retry if a key is missing. The Telegram Checklist Generator coerces raw LLM text directly into a typed array field with no JSON-parse safety net.

6 What remains blocked, and why

The top-level README is direct about what is and is not in this folder. Most of the originally-blocked automations were later unblocked once the owner enabled per-workflow MCP (Model Con-

text Protocol) access in n8n’s own settings, which let the tooling used to build this Portfolio actually read those workflows’ node graphs directly from the live instance. That unblocked, and later built: `linkedin-extractor-from-google-maps`, `submagic-video-clip-generator`, `heygen-ai-video-production`, `instagram-posting-workflow`, `instagram-link-finder-outreach`, `fathom-native-node-integration`, `gmail-outreach-follow-ups`, `calcom-trigger-workflow`, and `subscriber-email-sequence`. Several of those folders’ own README Challenges sections disclose real defects found only once the actual node graph could finally be read: dead ends, disabled triggers, duplicate logic, the same categories described in Section 5.

Five automations remain not built, by the owner’s own choice, because MCP access was never enabled for them and their source is not otherwise recoverable (not in the 2025-11-15 full backup export, not found on disk):

- Fathom.ai Workflow with Webhook Node (a sibling to the native-node version that *is* built, in `fathom-native-node-integration/`).
- Submagic – Video Clip Generator, the main production build (a sibling to a demo version that is built).
- Submagic Testing.
- Heygen – AI Video Production, the main production build (a sibling to a demo version that is built).
- A client-named scripting/agent automation, name withheld here, consistent with the collection’s anonymization policy.

To complete any of these later, the stated fix is the same one that already worked for the other nine: enable MCP access on that specific workflow in n8n’s settings.

Two more approved automations were only *partially* recoverable. The second “Instant VAPI Call” workflow (a sibling to the one built in `vapi-call-trigger-utility/`) and two sibling variants of the Instagram Reel Analyzer (siblings to the one built and documented in `instagram-reel-analyzer/`) have no accessible source anywhere checked. Each of those two folders documents plainly the one variant that could actually be verified, and states what is missing rather than guessing at it.

7 Closing notes and honest limits of this document

This Deep Dive covers 8 automations in real architectural detail out of 59, chosen to span every category in the collection and to include its largest workflow (190 nodes), its clearest orchestrator pattern, and its most candidly self-critical defect disclosures. It does not claim node-by-node coverage of the other 51 folders; where this document names an automation only in passing (Section 3’s category list), that is a folder-name-level reference, not a reviewed architecture.

Every diagram in this document is a simplified redraw using real node names and real connection order taken directly from the source README’s own Architecture section; no node name, defect, or metric in this document was invented. Where a README itself states that something is inferred rather than confirmed (the Cold Email system’s research-to-orchestrator handoff, the RAG pipeline’s Word-document re-trigger dependency), this document repeats that same uncertainty rather than resolving it into a false certainty.